Lasyankali Frontend Design Specification

AI-agent-ready product, experience, visual, interaction and implementation authority

Brand	Lasyankali
Product	Classical arts academy management web application
Primary users	Administrators, teachers, students and parents
Design target	Responsive, accessible and recruiter-ready frontend

Document purpose

This document is the design authority for an AI agent building the Lasyankali web application frontend. It defines the product intent, information architecture, visual system, interaction rules, responsive behavior, accessibility requirements, page states, payment journey, implementation boundaries, and acceptance criteria.

The agent must treat this document as a functional specification. It must not invent backend capabilities, silently change API contracts, expose unfinished AI features, or replace usability with decorative effects.

1 Product definition

1.1 Product summary

Lasyankali is a multidisciplinary classical arts academy management application. It supports dance, vocal music, instrumental arts, yoga, teachers, students, parents, batches, attendance, events, examinations, fees, and online payments.

The application has four authenticated roles:

- ADMIN
- TEACHER
- STUDENT
- PARENT

The product must feel culturally grounded, expressive, welcoming, disciplined, and modern. It is an operational application, so forms, tables, payments, attendance, and status information must remain clear and efficient.

1.2 Main goals

1. Reduce the effort required to operate the academy.
2. Give teachers fast access to assigned classes and attendance tasks.
3. Give students and parents clear access to schedules, progress, events, results, fees, and payment.

4. Establish a recognizable Lasyankali digital identity.
5. Demonstrate strong frontend engineering, accessibility, responsive design, API integration, and payment handling to recruiters.
6. Create a stable foundation for a later AI module.

1.3 Non-goals for the first frontend release

- Do not implement AI functionality yet.
- Do not add self-registration unless the backend supports it.
- Do not add password recovery unless the backend supports it.
- Do not invent chat, notification, reporting, export, or analytics APIs.
- Do not place sensitive keys, secrets, database information, or provider secrets in frontend code.
- Do not use frontend visibility as a substitute for backend authorization.

2 Agent operating rules

2.1 Required behavior

- Inspect the available backend contract before implementing a connected screen.
- Use the exact API paths, HTTP methods, request fields, enum values, and response fields already provided by the backend.
- Create one centralized API layer. Components must not scatter raw URLs throughout the project.
- Create reusable primitives and domain components before duplicating markup.
- Implement loading, empty, success, validation, forbidden, not-found, network-failure, and unexpected-error states.
- Preserve user input after recoverable submission errors.
- Keep role navigation and route guards consistent with backend authorization.
- Use realistic fictional demonstration data when the API is unavailable. Clearly isolate mock data so it cannot accidentally ship as production behavior.
- Build mobile behavior while building desktop behavior, not afterward.
- Keep the code understandable for an intern who will maintain and explain it.

2.2 Prohibited behavior

- Do not redesign backend entities from the frontend.
- Do not fabricate successful responses.
- Do not mark a payment successful before backend verification.
- Do not show raw stack traces, provider errors, SQL messages, or enum underscores to users.
- Do not use color as the only status indicator.
- Do not add animation to every component.
- Do not use autoplay video backgrounds, heavy parallax, constant floating objects, or large decorative loaders.
- Do not use culturally sensitive symbols merely as decoration.
- Do not make dance the only visible discipline; the product is multidisciplinary.
- Do not expose future AI navigation as a working production feature.

3 Target audiences

Role	Main context	Main goals	Design response
Administrator	Frequent desktop use and high data volume	Manage users, batches, assignments, attendance, events, exams, fees, and payments	Efficient tables, filters, quick actions, clear status, predictable forms
Teacher	Phone, tablet, or laptop before and during class	View assigned batches, students, attendance, events, and permitted academic work	Fast batch switching, large touch targets, clear save confirmation
Student	Mostly mobile and periodic use	View profile, batch, attendance, events, results, and fees; pay fees	Simple navigation, clear deadlines, friendly summaries, safe payment flow
Parent	Mostly mobile; may manage children	Review child information, attendance, results, and fees; pay safely	Child context, plain language, reassuring payment states
Recruiter	Short desktop walkthrough	Understand product quality and technical breadth	Strong first impression, coherent flows, responsive behavior, meaningful motion

4 Brand and art direction

4.1 Brand character

The visual language must combine:

- Classical discipline
- Contemporary product design
- Multidisciplinary arts
- Warmth and hospitality
- Controlled color
- Professional operational clarity

The interface should feel connected to rhythm, movement, music, practice, and stage craft. It must not look like a generic blue school dashboard or a decorative event poster.

4.2 Logo requirement

The final Lasyankali logo is the authority for exact brand colors and logo clear space. Until the logo file is supplied:

- Use a replaceable logo component.
- Do not redraw or approximate the logo.
- Keep a text fallback reading Lasyankali.
- Store logo assets in a dedicated brand asset directory.
- Use the provisional tokens below.

4.3 Provisional color tokens

```
:root {
  --color-brand-900: #35173F;
  --color-brand-700: #5A2A6E;
  --color-brand-500: #7C4A8D;
  --color-accent-gold: #E8A317;
  --color-accent-teal: #147D73;
  --color-accent-rose: #C65D7B;

  --color-bg: #FFF9F0;
  --color-surface: #FFFFFF;
  --color-surface-soft: #F7F2F8;
  --color-border: #DDD4E0;
  --color-text: #25212A;
  --color-text-muted: #6B6470;

  --color-success: #2E7D57;
  --color-warning: #A96300;
  --color-danger: #B3261E;
  --color-info: #276A9B;
}
```

Rules:

- Primary actions use brand 700 with accessible white text.
- Gold is an accent, not a body-text color on light backgrounds.
- Teal and rose may distinguish selected art categories but must not redefine shared component behavior.
- Destructive actions use danger styling and explicit language.
- Status must always include readable text and may include an icon.
- Validate every final combination against WCAG 2.2 AA contrast.

4.4 Typography

- Use one interface sans-serif family: Inter, Manrope, or Noto Sans.
- Use an elegant readable serif such as Noto Serif Display only for selected welcome headings, page introductions, or marketing-style moments.
- Use sans-serif type for navigation, forms, tables, statuses, fees, and payment.
- Avoid calligraphic fonts in functional UI.
- Support Unicode and future Indian-language scripts.

Recommended scale:

Token	Desktop	Mobile	Use
Display	48 px	34 px	Rare welcome or public-page heading
H1	32 px	26 px	Page title
H2	24 px	21 px	Major page section
H3	19 px	18 px	Card or subsection heading
Body	16 px	16 px	Default text
Small	14 px	14 px	Secondary information
Caption	12 px	12 px	Carefully used metadata

Use line heights between 1.4 and 1.6 for body text. Do not reduce mobile body text below 16 px in forms.

4.5 Imagery and motifs

- Prefer authentic academy photography with permission.

- Represent dance, vocal music, instruments, percussion, and yoga where appropriate.
- Use restrained line motifs inspired by movement arcs, rhythm marks, stage geometry, strings, percussion, mudra geometry, and ghungroo patterns.
- Keep motifs low contrast and away from forms, tables, and long text.
- Use illustration primarily for empty states, onboarding moments, and dashboard welcome areas.
- Avoid unverified costume details, religious decoration, generic stock yoga imagery, and cultural stereotypes.

4.6 Shape, elevation, and spacing

```
:root {
  --space-1: 4px;
  --space-2: 8px;
  --space-3: 12px;
  --space-4: 16px;
  --space-6: 24px;
  --space-8: 32px;
  --space-12: 48px;
  --space-16: 64px;

  --radius-sm: 8px;
  --radius-md: 12px;
  --radius-lg: 16px;
  --radius-pill: 999px;

  --shadow-raised: 0 8px 24px rgba(53, 23, 63, 0.10);
  --shadow-overlay: 0 18px 50px rgba(37, 33, 42, 0.18);
}
```

Use subtle borders and spacing for ordinary cards. Reserve stronger shadows for menus, drawers, dialogs, and floating overlays. Do not make every control pill-shaped.

4.7 Motion

- Use 150 to 250 ms for common hover, focus, menu, and state transitions.
- Use one controlled dashboard entrance sequence, not independent animation on every card.
- Use motion for orientation, expansion, progress, successful save, and payment confirmation.
- Do not delay task completion for animation.
- Respect `prefers-reduced-motion` and provide a static equivalent.
- Avoid flashing, looping decorative motion, and large layout shifts.

5 Information architecture

5.1 Global application areas

```
Public
  Login
  Access help if supported
  Privacy and support information if supplied

Authenticated shared
  Role dashboard
  Profile
  Session and logout
  Error and access-denied pages

Admin
  Dashboard
  Students and parents
  Teachers
  Batches
  Student and teacher assignments
  Attendance
  Events
  Exams and results
  Fees
  Payments

Teacher
  Dashboard
  Assigned batches
  Batch students
  Attendance
  Permitted events and examination work

Student
  Dashboard
  My profile
  My batch
  Attendance
  Events
  Exams and results
  My fees
  Payment

Parent
  Dashboard
  Child context
  Attendance
  Events
  Exams and results
  Child fees
  Payment

Future
  AI assistant or insights after separate approval
```

5.2 Navigation behavior

Desktop:

- Persistent left sidebar.
- Compact top bar with role identity, profile entry, and logout.
- Breadcrumbs on deeper pages.
- Page header with title, concise support text, and one primary action.

Tablet:

- Collapsible navigation rail or drawer.
- Preserve page title, filters, and primary action.

Mobile:

- Top bar with menu drawer, or a small bottom navigation for the highest-frequency student and parent tasks.
- Do not place all admin destinations in bottom navigation.
- Keep primary actions reachable without horizontal scrolling.
- Convert suitable tables into record cards; preserve horizontal scrolling only when column comparison is necessary.

Navigation rules:

- Use task labels such as My Fees, Record Attendance, and Assign Students.
- Hide destinations unavailable to the current role.
- Route guards improve usability but do not replace backend authorization.
- After refresh, restore the authenticated route when the session remains valid.
- On session expiry, return to login with a readable explanation.

6 Route and page matrix

Use these as frontend route intentions. Adjust path names to the established project convention without changing the page ownership.

Page	Role	Priority	Main content
Login	Public	P0	Credentials, validation, loading, invalid-login feedback
Dashboard	All authenticated	P0	Role-specific summary and primary actions
Students list	ADMIN	P0	Search, filters, pagination, create, detail navigation
Student detail	ADMIN	P0	Profile, parent, batch, attendance, exams, fees
Teachers list and detail	ADMIN	P0	Profile and assigned batches
Batches list and detail	ADMIN, TEACHER as allowed	P0	Discipline, level, schedule, capacity, teacher, students
Assignment	ADMIN	P0	Assign student or teacher with capacity and duplicate feedback
Attendance	ADMIN, TEACHER	P0	Session context, roster, marking, save feedback
Admin fees	ADMIN	P0	Filters, monthly generation, statuses, results
My fees	STUDENT	P0	Pending, overdue, paid, amount, due date, Pay action
Child fees	PARENT	P0	Child context and fee/payment information
Payment flow	STUDENT, PARENT	P0	Order creation, Checkout, verification, result
Events	Relevant roles	P1	List, detail, registration or management supported by API
Exams and results	Relevant roles	P1	Schedule, registration, unpublished and published states
Payments list	ADMIN	P1	Created, success, failed, provider references, filters
Profile	All authenticated	P1	Identity and permitted account information
Notifications	All authenticated	P2	Only if backend support exists
AI assistant	Approved roles	Future	Do not implement in this phase

7 Core user flows

7.1 Login

1. User opens Login.
2. User enters username and password.
3. Frontend validates required fields.
4. Frontend submits to the existing authentication API.
5. On success, frontend stores the JWT according to the chosen security strategy and obtains role context.
6. User is sent to the correct role dashboard.
7. Protected API requests include `Authorization: Bearer <token>`.
8. Invalid credentials produce an inline form-level error.
9. Expired authentication returns the user to Login and explains that the session ended.

7.2 Admin creates and assigns a student

1. Admin opens Students.
2. Admin creates student and parent details using supported backend fields.
3. Validation appears near each field and in a summary when necessary.
4. Successful submission opens or links to the created student.
5. Admin selects Assign Batch.
6. The interface shows batch name, discipline, level, timing, capacity, and current status.
7. Admin confirms assignment.
8. The result appears on student and batch detail screens.

7.3 Teacher records attendance

1. Teacher opens an assigned batch.
2. The interface shows the selected date and attendance session.
3. The roster shows enrolled students with large Present and Absent controls.
4. The interface identifies unmarked students before submission.
5. While saving, duplicate submission is disabled.
6. Success confirms the batch, date, and number of saved records.

7.4 Admin generates fees

1. Admin opens Fees.
2. Admin chooses the billing month.
3. The interface explains that the month represents a monthly bill.
4. Admin starts generation.
5. The interface displays created and skipped counts when the backend provides them.
6. Fee records refresh with billing month, amount, due date, student, batch, and status.

7.5 Student or parent pays a fee

1. User opens My Fees or Child Fees.
2. User selects a Pending or Overdue fee.

3. The interface shows fee amount, billing month, due date, student, and batch.
4. User chooses Pay.
5. Frontend calls the payment-order endpoint with the fee ID and JWT.
6. Backend returns local payment ID, provider order ID, amount in paise, currency, and public Razorpay key ID.
7. Frontend opens Razorpay Checkout using those returned values.
8. User completes, cancels, or fails the provider flow.
9. On successful provider response, frontend sends provider order ID, provider payment ID, and provider signature to the backend verification endpoint.
10. The frontend does not calculate or validate the signature itself.
11. During verification, repeated actions are disabled and progress is announced.
12. On verified success, the frontend refreshes fee data and shows Paid.
13. On cancellation or failure, the fee remains Pending or Overdue and the interface offers a safe retry.

Payment safety rules:

- Never include the Razorpay secret in frontend code.
- The public key ID may be passed to Checkout.
- Never mark payment Success based only on the Checkout callback.
- Backend verification is the source of truth.
- Do not expose raw provider exceptions to users.
- Preserve provider references for support displays only when appropriate.

8 Shared application shell

8.1 Desktop shell

- Sidebar width: approximately 248 px expanded and 72 px collapsed.
- Top bar height: approximately 64 px.
- Content padding: 24 to 32 px.
- Main content uses a fluid width with a reasonable maximum for forms and prose.
- Data-heavy pages may use the available width.

Sidebar order:

1. Brand area
2. Role-specific primary navigation
3. Secondary navigation or support if implemented
4. User/profile/logout area

8.2 Mobile shell

- Header height: approximately 56 px.
- Menu opens as an accessible drawer.
- Drawer traps focus while open and returns focus to its trigger when closed.
- Important student and parent destinations may use a maximum of four bottom-navigation items.
- Page content uses 16 px side padding.

8.3 Page header pattern

Every page header contains:

- Breadcrumb where useful
- H1 page title
- One short description when clarification is needed
- One primary action
- Secondary actions in a menu or secondary button

Avoid several equally prominent buttons.

9 Component system

9.1 Primitive components

- Button: primary, secondary, text, destructive, icon, loading
- Input: text, password, email, telephone, number, date, time, search
- Select and multi-select where supported
- Checkbox, radio, switch
- Textarea
- Badge and status label
- Avatar
- Tooltip
- Divider
- Skeleton
- Spinner for small localized progress only

Each interactive primitive requires:

- Default
- Hover
- Focus-visible
- Pressed or selected
- Disabled
- Loading when relevant
- Valid and invalid when relevant

9.2 Composite components

- AppShell
- Sidebar
- TopBar
- MobileDrawer
- PageHeader
- Breadcrumbs
- FilterBar

- DataTable
- MobileRecordCard
- Pagination
- EmptyState
- ErrorState
- ConfirmationDialog
- FormSection
- DetailSection
- SummaryCard
- StatCard
- Tabs
- Toast or notification region

9.3 Domain components

- DisciplineMarker
- BatchCard
- CapacityIndicator
- StudentIdentity
- ParentIdentity
- TeacherIdentity
- AttendanceRoster
- AttendanceStatusControl
- ExamCard
- EventCard
- FeeCard
- FeeStatus
- PaymentStatus
- PaymentSummary
- ChildSelector

9.4 Status language

Display code values as readable labels:

- ACTIVE -> Active
- INACTIVE -> Inactive
- PENDING -> Pending
- PAID -> Paid
- OVERDUE -> Overdue
- CREATED -> Created
- SUCCESS -> Successful
- FAILED -> Failed

Status labels must include text. Icons and color are supplementary.

10 Screen specifications

10.1 Login

Desktop:

- Balanced split layout is allowed.
- One side may use brand imagery or subtle art motifs.
- The form side remains quiet and focused.

Mobile:

- Form appears first.
- Artwork becomes a compact brand header.

Required elements:

- Replaceable logo
- Welcome heading
- Username label and field
- Password label and field
- Show-password control
- Primary Login button
- Error region that does not move the entire layout unexpectedly
- Loading state

Do not display unsupported sign-up or forgot-password links.

10.2 Role dashboards

Admin dashboard:

- Today context
- Two to four meaningful totals
- Upcoming classes or events
- Fee summary if supported
- Recent activity if supported
- Quick actions: Add Student, Create Batch, Assign People, Generate Fees

Teacher dashboard:

- Today assigned batches
- Attendance completion status
- Upcoming event or exam information
- Primary action: Open Batch or Record Attendance

Student dashboard:

- Next class
- Attendance summary
- Upcoming event or exam

- Unpaid fee summary
- Primary actions: View Schedule, View Results, Pay Fee

Parent dashboard:

- Selected child identity
- Child switcher only when supported
- Next class and attendance information
- Unpaid fee summary
- Primary actions: Review Progress, View Fees, Pay Fee

Dashboard rules:

- Do not use vanity metrics.
- Charts must answer a clear question.
- Include loading skeleton, empty data, partial data, and error states.
- Avoid more than one dominant primary action.

10.3 List pages

Page structure:

1. Page header
2. Search and essential filters
3. Result count and optional active-filter summary
4. Table on desktop
5. Record cards or adaptive table on mobile
6. Pagination when required

Required states:

- Initial loading
- Results
- No records
- No results for current filters
- Network failure with Retry
- Permission failure

Row actions must use consistent placement and language.

10.4 Detail pages

- Start with identity summary and status.
- Group information by meaning.
- Show related records in sections or a restrained number of tabs.
- Keep destructive actions separate from primary actions.
- Ask for confirmation before destructive or irreversible operations.
- Do not hide frequently needed information inside nested accordions.

Student detail should be prepared for:

- Personal information

- Parent information
- Batch assignment
- Attendance
- Exams and results
- Fees

10.5 Forms

- Use visible labels; placeholders do not replace labels.
- Mark required fields consistently.
- Group related fields with headings.
- Use one column on mobile.
- Use no more than two logical columns on wide screens.
- Show field errors beside fields.
- Use a form-level summary when several fields fail.
- Retain valid user input after server errors.
- Disable duplicate submission while saving.
- On successful creation, show confirmation and take the user to the created record or updated list.

10.6 Attendance

- Show batch, date, session, teacher, and roster context.
- Present and Absent controls must be fast to use and keyboard accessible.
- Do not depend on green and red alone.
- Clearly identify unmarked students.
- Provide a review step or count before save when many records exist.
- Confirm saved count and date.
- Avoid accidental overwriting when editing existing attendance.

10.7 Fees

Admin fee list columns or record fields:

- Student
- Batch
- Billing month
- Amount
- Due date
- Status
- Updated date when useful

Student or parent fee card:

- Student identity when parent context is used
- Batch name
- Billing month
- Amount in INR

- Due date
- Status label
- Pay action only when allowed

Fee states:

- Pending
- Overdue
- Paid
- Empty
- Order creation loading
- Payment verification loading
- Payment successful
- Payment cancelled or failed

10.8 Payment result

Successful state includes:

- Clear success heading
- Amount
- Student or fee context
- Payment date if returned
- Provider payment reference when appropriate
- Button to return to fees

Failed or cancelled state includes:

- Plain explanation
- Confirmation that the fee was not marked Paid when verification failed
- Retry option when safe
- Support direction when retry is not appropriate

11 Data presentation rules

- Use tables for comparison and repeated records.
- Use cards for mobile summaries.
- Align amounts consistently.
- Display INR as `₹500.00` where decimals are useful.
- Display dates to users as `26 Sep 2026` or another single approved format.
- Keep API date values in the format required by the backend.
- Do not display database column names or raw enum values.
- Truncate only when the full value remains available through wrapping, expansion, or an accessible tooltip.
- Charts require labels and a text summary.
- Do not use a chart when a number or short list communicates the information more clearly.

12 Feedback and error handling

12.1 Feedback hierarchy

- Field message: error attached to one input.
- Form message: submission-wide validation or authentication failure.
- Inline page message: data loading or permission problem affecting a section.
- Toast: short confirmation for completed reversible actions.
- Dialog: confirmation or information that requires a decision.
- Dedicated result page: payment success or serious workflow completion.

12.2 Error language

Error messages must explain:

1. What happened.
2. Whether the action completed.
3. What the user can do next.

Examples:

- `Choose a billing month before generating fees.`
- `You do not have access to this fee.`
- `The payment could not be verified. The fee was not marked Paid. Try again or contact the academy.`

Do not show `NullPointerException`, `AccessDeniedException`, Razorpay stack traces, or database errors.

13 Responsive specification

Range	Width	Main behavior
Small mobile	320-479 px	Single column, compact brand artwork, drawer navigation, large controls
Large mobile	480-767 px	Single column with improved spacing and selected two-column summaries
Tablet	768-1023 px	Collapsible navigation, two-column cards, adaptive tables
Desktop	1024-1439 px	Persistent sidebar, multi-column dashboard, full tables
Large desktop	1440 px and above	Bounded content and comfortable data layouts without stretched cards

Responsive requirements:

- No essential action may require horizontal page scrolling.
- Touch targets should be at least 44 by 44 CSS pixels wherever practical.
- Dialogs may become full-screen panels or bottom sheets on narrow screens.
- Focus order follows the visual order.
- Tables may scroll horizontally only when comparison requires it.
- Mobile cards show the most important two or three values, status, and one primary action.
- Test at 320 px width and 200 percent zoom.

14 Accessibility specification

Target WCAG 2.2 Level AA.

14.1 Required implementation

- Use semantic landmarks: header, nav, main, aside, and footer where appropriate.
- Use one H1 per page and logical heading levels.
- Ensure all controls have programmatic accessible names.
- Associate errors and supporting text with form fields.
- Make every action keyboard usable.
- Provide visible focus treatment with sufficient contrast.
- Trap focus in modal dialogs and restore it on close.
- Provide a skip-to-main-content link.
- Use `aria-live` carefully for async save, verification, and error feedback.
- Add meaningful alternative text to informative images.
- Mark decorative motifs as decorative.
- Do not communicate status using color alone.
- Support reduced motion.
- Maintain readable reflow at 200 percent zoom.
- Announce payment Checkout opening, verification progress, success, and failure.

14.2 Contrast targets

- Normal text: at least 4.5:1.
- Large text: at least 3:1.
- Meaningful non-text graphics and control boundaries: follow applicable 3:1 guidance.
- Focus indicator must remain clear against adjacent colors.

14.3 Accessibility testing

- Keyboard-only test every P0 flow.
- Test Login, navigation, forms, attendance, and fees with a screen reader.
- Run automated checks, then perform manual review.
- Verify reduced-motion behavior.
- Verify error identification without color.

15 Content system

15.1 Voice

Use language that is:

- Respectful
- Encouraging
- Direct
- Calm
- Specific

Avoid ceremonial language for routine administration and avoid technical terms when ordinary language works.

15.2 Content rules

- Use sentence case.
- Buttons start with verbs: Add Student, Assign Batch, Generate Fees, Pay ₹500.
- Keep confirmations specific: `Attendance saved for Beginner Evening Batch.`
- Display readable enum labels.
- Use one date format consistently.
- Avoid unexplained abbreviations.
- Do not claim AI behavior before it exists.

15.3 Required content assets

- Final logo in SVG and transparent PNG
- Official color and typography guidance if available
- Academy description and contact details
- Privacy and payment-support content
- Approved discipline names
- Authentic photography with usage permission
- Empty-state and support messages
- Fictional portfolio-safe sample data

16 Frontend architecture expectations

The agent should use the frontend stack approved by the project owner. If React with Vite is selected, use this conceptual structure:

```
src/
  app/
    router/
    providers/
    guards/
  api/
    client/
    auth/
    students/
    teachers/
    batches/
    attendance/
    events/
    exams/
    fees/
    payments/
  assets/
    brand/
    icons/
    images/
  components/
    primitives/
    layout/
    feedback/
    data-display/
    domain/
  features/
    auth/
    dashboard/
    students/
    teachers/
    batches/
    attendance/
    events/
    exams/
```

```
    fees/
    payments/
  hooks/
  styles/
    tokens/
    global/
  utils/
```

Architecture rules:

- Separate API calls from presentation components.
- Centralize JWT attachment and standard error conversion.
- Keep role logic in guards and navigation configuration.
- Avoid one enormous global context containing all server data.
- Use feature-level modules.
- Keep shared components free from feature-specific assumptions.
- Use route-level lazy loading where it improves initial load.
- Do not log JWTs, passwords, payment signatures, or secrets.

17 API integration behavior

17.1 Standard request behavior

- Show localized loading feedback.
- Prevent accidental duplicate submission.
- Convert HTTP errors into user-safe messages.
- Handle 401 by clearing invalid authentication and returning to Login.
- Handle 403 with a dedicated access-denied state.
- Handle 404 with a not-found state.
- Handle validation responses beside the correct fields when possible.
- Allow Retry for safe idempotent reads.

17.2 Server state refresh

After successful mutation:

- Refresh the affected list or detail data.
- Do not assume the local optimistic value is the database result for payment or security-sensitive changes.
- Payment verification must refresh both payment and fee state.
- Assignment must refresh student and batch relationships.
- Attendance save must refresh the saved session or show the returned saved result.

17.3 Authentication state

- Keep authenticated identity and role available to route guards and navigation.
- Avoid decoding authorization decisions solely from untrusted frontend state.
- Clear authentication on explicit logout.
- Explain session expiry instead of silently failing.

18 Performance requirements

- Optimize and size images for their display context.
- Lazy-load feature routes where suitable.
- Avoid loading every management dataset on the dashboard.
- Use stable skeleton dimensions to reduce layout shifts.
- Debounce search when API-backed search requires it.
- Paginate or virtualize only when data volume justifies it.
- Avoid expensive animation on large tables.
- Test on mobile throttling and slower networks.

19 Prototype and usability tests

The design or implementation must support these review tasks:

Participant	Task	Success evidence
Admin	Create a student and assign an active batch	Completes without help and confirms assignment
Teacher	Record attendance for an assigned batch	Understands date, roster, incomplete marks, and save result
Student	Find an unpaid fee and begin payment	Identifies correct fee, amount, and due date
Parent	Find a child's fee and begin payment	Understands selected child and ownership context
Admin	Generate monthly fees and review results	Understands created and skipped outcomes

Review questions:

- Can the user identify the current location?
- Is the next important action obvious?
- Can every status be understood without color?
- Can mobile users complete the task without precision tapping?
- Can users recover from validation, network, and permission errors?
- Does the product feel connected to classical arts while remaining operationally credible?
- Can a developer identify every component state and responsive behavior?

20 Recruiter-facing requirements

The frontend should demonstrate:

- A coherent design system
- Role-based routing and navigation
- Reusable component architecture
- Real backend API integration
- Authentication and authorization-aware UX
- Responsive desktop, tablet, and mobile design

- Accessible forms and keyboard interaction
- Loading, empty, error, and success states
- Data tables and adaptive mobile records
- Payment order and verification flow
- Restrained, meaningful animation
- Clear technical organization

The first 30 seconds of a demonstration should make the brand, user role, main task, and product quality understandable.

21 Definition of done

A screen is not complete merely because it looks correct with ideal data. It is complete only when:

- It uses approved design tokens.
- It works at required breakpoints.
- It supports keyboard navigation.
- It has visible focus.
- It has loading, empty, error, and success states.
- It displays realistic content lengths.
- It uses centralized API handling.
- It respects role access.
- It does not expose sensitive information.
- It has been tested with the backend response.
- It has no console errors.
- It has no obvious accessibility violations.

A P0 user journey is complete only when the user can finish it from the UI through the backend and see the correct database-backed result.

22 Delivery sequence

Build in this order:

1. Design tokens and global styling
2. Reusable primitives
3. Application shell and responsive navigation
4. Authentication and route protection
5. Role dashboards
6. Students and parents
7. Teachers
8. Batches and assignments
9. Attendance
10. Events
11. Exams and results
12. Fees

13. Razorpay Checkout and verification
14. Final accessibility and responsive pass
15. End-to-end testing
16. Deployment preparation
17. Separate AI design and implementation phase

Each module must be tested before the next dependent module begins.

23 Open decisions

The agent must not silently choose these. Record the decision before final implementation:

- Final logo and official color values
- Approved interface font family
- Exact frontend stack and styling method
- Exact route naming convention
- Token storage strategy
- Backend-supported teacher permissions
- Whether one parent can manage multiple children
- Event and examination permissions per role
- Privacy and support content
- First approved AI use case

24 Final instruction to the AI agent

Build Lasyankali as an arts-centered operational product. It should be expressive enough to be memorable, structured enough to support real academy work, and clear enough that administrators, teachers, students, and parents can complete important tasks confidently.

Prioritize correctness, usability, accessibility, and maintainable components. Use color, art motifs, and motion with restraint. Never allow decoration to obscure task completion, payment status, attendance, or user data.

25 Reference material

- Fullstory UI design principles: https://www.fullstory.com/blog/ui-design/
- Blush UI design guide: https://blush.design/blog/post/guide-ui-design
- Konrad UI design guide: https://www.konrad.com/research/ui-design-example
- UX Design Institute UI process: https://www.uxdesigninstitute.com/blog/guide-to-the-ui-design-process/
- W3C WCAG 2.2 Quick Reference: https://www.w3.org/WAI/WCAG22/quickref/